In some situations a maximum is not required and matching scores must instead be compared to a threshold. This can be done using an array of hard-limiting nodes with internal thresholds set to the desired threshold values. Outputs of these nodes will be −1 unless the inputs exceed the threshold values. Alternatively, thresholds could be set adaptively using a common inhibitory input fed to all nodes. This threshold could be ramped up or down until the output of only one node was positive.

THE CARPENTER/GROSSBERG CLASSIFIER

Carpenter and Grossberg [3], in the development of their Adaptive Resonance Theory have designed a net which forms clusters and is trained without supervision. This net implements a clustering algorithm that is very similar to the simple sequential leader clustering algorithm described in [16]. The leader algorithm selects the first input as the exemplar for the first cluster. The next input is compared to the first cluster exemplar. It "follows the leader" and is clustered with the first if the distance to the first is less than a threshold. Otherwise it is the exemplar for a new cluster. This process is repeated for all following inputs. The number of clusters thus grows with time and depends on both the threshold and the distance metric used to compare inputs to cluster exemplars.

The major components of a Carpenter/Grossberg classification net with three inputs and two output nodes is presented in Fig. 10. The structure of this net is similar to that of the Hamming net. Matching scores are computed using feed-forward connections and the maximum value is enhanced using lateral inhibition among the output nodes. This net differs from the Hamming net in that feedback connections are provided from the output nodes to the input nodes. Mechanisms are also provided to turn off that output node with a maximum value, and to compare exemplars to the input for the threshold test required by the leader algorithm. This net is completely described using nonlinear differential equations, includes extensive feedback, and has been shown to be stable [3]. In typical operation, the differential equations can be shown to implement the clustering algorithm presented in Box 3.

Box 3. Carpenter/Grossberg Net Algorithm

Step 1. Initialization

$$t_{ij}(0) = 1$$

$$b_{ij}(0) = \frac{1}{1 + N}$$

$$0 \le i \le N - 1,$$

$$0 \le j \le M - 1$$

Set ρ, $\quad 0 \le \rho \le 1$

In these equations $b_{ij}(t)$ is the bottom up and $t_{ij}(t)$ is the top down connection weight between input node i and output node j at time t. These weights define the exemplar specified by output node j. The fraction ρ is the vigilance threshold which indicates how close an input must be to a stored exemplar to match.

Step 2. Apply New Input

Step 3. Compute Matching Scores

$$\mu_j = \sum_{i=0}^{N-1} b_{ij}(t)x_i \qquad 0 \le j \le M - 1$$

In this equation μ_j is the output of output node j and x_i is element i of the input which can be 0 or 1.

Step 4. Select Best Matching Exemplar

$$\mu_{j^*} = \max_j \{\mu_j\}$$

This is performed using extensive lateral inhibition as in the maxnet.

Step 5. Vigilance Test

$$\|X\| = \sum_{i=0}^{N-1} x_i$$

$$\|T \cdot X\| = \sum_{i=0}^{N-1} t_{ij^*} x_i$$

is $\dfrac{\|T \cdot X\|}{\|X\|} > \rho$?

NO → GO TO STEP 6

YES → GO TO STEP 7

Step 6. Disable Best Matching Exemplar

The output of the best matching node selected in Step 4 is temporarily set to zero and no longer takes part in the maximization of Step 4. Then go to Step 3.

Step 7. Adapt Best Matching Exemplar

$$t_{ij^*}(t + 1) = t_{ij^*}(t)x_i$$

$$b_{ij^*}(t + 1) = \frac{t_{ij^*}(t)x_i}{.5 + \sum_{i=0}^{N-1} t_{ij^*}(t)x_i}$$

Step 8. Repeat by Going to Step 2

(First enable any nodes disabled in Step 6)

The algorithm presented in Box 3 assumes that "fast learning" is used as in the simulations presented in [3] and thus that elements of both inputs and stored exemplars take on only the values 0 and 1. The net is initialized by effectively setting all exemplars represented by connection weights to zero. In addition, a matching threshold called *vigilance* which ranges between 0.0 and 1.0 must be set. This threshold determines how close a new input pattern must be to a stored exemplar to be considered similar. A value near one requires a close match and smaller values accept a poorer match. New inputs are presented sequentially at the bottom of the net as in the Hamming net. After presentation, the input is compared to all stored exemplars in parallel as in the Hamming net to produce matching scores. The exemplar with the highest matching score is selected using lateral inhibition. It is then compared to the input by computing the ratio of the dot product of the input and the best matching exemplar (number of 1 bits in common) divided by the number of 1 bits in the input. If this ratio is greater than the vigilance threshold, then the input is considered to be similar to the best matching exemplar and that exemplar is updated by performing a logical AND operation between its bits and those in the input. If the ratio is less than the vigilance threshold, then the input is considered to be different from all exemplars and it is added as a new exemplar. Each additional new exemplar requires one node and 2N connections to compute matching scores.

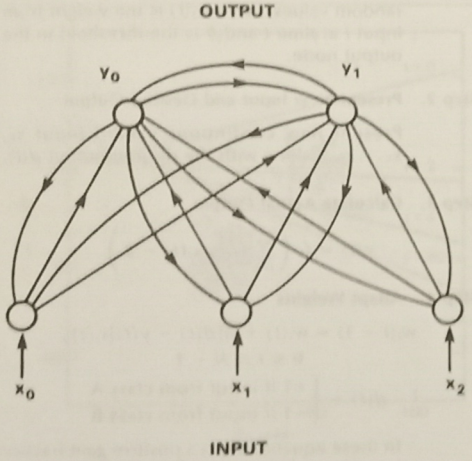


Figure 10. The major components of the Carpenter Grossberg Classification net. A binary input is presented at the bottom and when classification is complete only one output is high. Not shown are additional components required to perform the vigilance test and to disable the output node with the largest output.

The behavior of the Carpenter/Grossberg net is illustrated in Fig. 11. Here it is assumed that patterns to be recognized are the three patterns of the letters "C", "E", and "F" shown in the left side of this figure. These patterns have 64 pixels each that take on the value 1 when black and 0 when white. Results are presented when the vigilance threshold was set to 0.9. This forces separate exemplar patterns to be created for each letter.

The left side of Fig. 11 shows the input to the net on successive trials. The right side presents exemplar patterns formed after each pattern had been applied. In this example "C" was presented first followed by "E" followed by "F", etc. After the net is initialized and a "C" is applied, internal connection weights are altered to form an internal exemplar that is identical to the "C". After an "E" is then applied, a new "E" exemplar is added. Behavior is similar for a new "F" leading to three stored exemplars. If the vigilance threshold had been slightly lower, only two exemplars would have been present after the "F"; one for "F" and one for both "C" and "E" that would have been identical to "C" pattern. Now, when a noisy "F" is applied

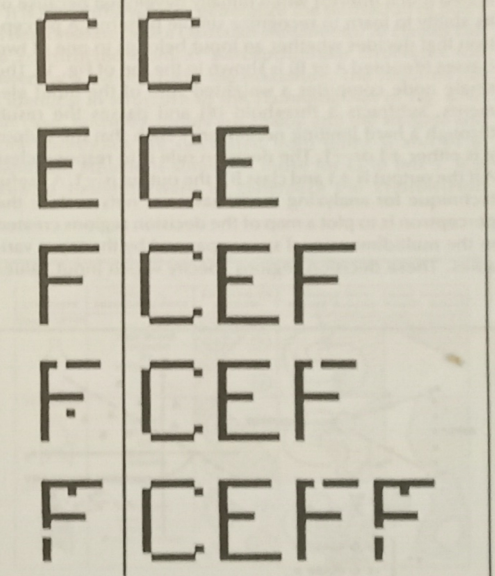


Figure 11. An example of the behavior of the Carpenter Grossberg net for letter patterns. Binary input patterns on the left were applied sequentially starting with the upper "C" pattern. Exemplars formed by top-down connection weights after each input was presented are shown at the right.

with a missing black pixel in the upper edge it is accepted as being similar to the "F" exemplar and degrades this exemplar due to the AND operation performed during updating. When another noisy "F" is applied again with only one black pixel missing, it is considered different from existing exemplars and a new noisy "F" exemplar is added. This will occur for further noisy "F" inputs leading to a growth of noisy "F" exemplars.

These results illustrate that the Carpenter/Grossberg algorithm can perform well with perfect input patterns but that even a small amount of noise can cause problems. With no noise, the vigilance threshold can be set such that the two patterns which are most similar are considered different. In noise, however, this level may be too high and the number of stored exemplars can rapidly grow until all available nodes are used up. Modifications are necessary to enhance the performance of this algorithm in noise. These could include adapting weights more slowly and changing the vigilance threshold during training and testing as suggested in [3].

SINGLE LAYER PERCEPTRON

The single layer perceptron [39] is the first of three nets from the taxonomy in Fig. 3 that can be used with both continuous valued and binary inputs. This simple net generated much interest when initially developed because of its ability to learn to recognize simple patterns. A perceptron that decides whether an input belongs to one of two classes (denoted A or B) is shown in the top of Fig. 12. The single node computes a weighted sum of the input elements, subtracts a threshold (θ) and passes the result through a hard limiting nonlinearity such that the output y is either +1 or −1. The decision rule is to respond class A if the output is +1 and class B if the output is −1. A useful technique for analyzing the behavior of nets such as the perceptron is to plot a map of the decision regions created in the multidimensional space spanned by the input variables. These decision regions specify which input values

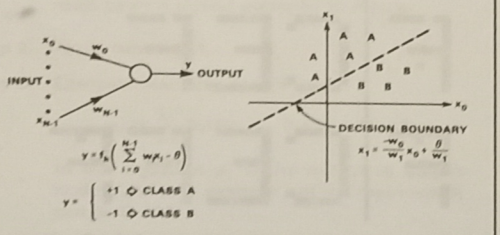


Figure 12. A single layer perceptron that classifies an analog input vector into two classes denoted A and B. This net divides the space spanned by the input into two regions separated by a hyperplane or a line in two dimensions as shown on the top right.

result in a class A and which result in a class B response. The perceptron forms two decision regions separated by a hyperplane. These regions are shown in the right side of Fig. 12 when there are only two inputs and the hyperplane is a line. In this case inputs above the boundary line lead to class A responses and inputs below the line lead to class B responses. As can be seen, the equation of the boundary line depends on the connection weights and the threshold.

Connection weights and the threshold in a perceptron can be fixed or adapted using a number of different algorithms. The original perceptron convergence procedure for adjusting weights was developed by Rosenblatt [39]. It is described in Box 4. First connection weights and the threshold value are initialized to small random non-zero values. Then a new input with N continuous valued elements is applied to the input and the output is computed as in Fig. 12. Connection weights are adapted only when an error occurs using the formula in step 4 of Box 4. This formula includes a gain term (η) that ranges from 0.0 to 1.0 and controls the adaptation rate. This gain term must be adjusted to satisfy the conflicting requirements of fast adaptation for real changes in the input distributions and averaging of past inputs to provide stable weight estimates.

Box 4. The Perceptron Convergence Procedure

Step 1. Initialize Weights and Threshold

Set $w_i(0)$ $(0 \leq i \leq N-1)$ and θ to small random values. Here $w_i(t)$ is the weight from input i at time t and θ is the threshold in the output node.

Step 2. Present New Input and Desired Output

Present new continuous valued input $x_0, x_1 \ldots x_{N-1}$ along with the desired output $d(t)$.

Step 3. Calculate Actual Output

$$y(t) = f_h\left(\sum_{i=0}^{N-1} w_i(t)x_i(t) - \theta\right)$$

Step 4. Adapt Weights

$$w_i(t+1) = w_i(t) + \eta[d(t) - y(t)]x_i(t),$$
$$0 \leq i \leq N-1$$

$$d(t) = \begin{cases} +1 \text{ if input from class A} \\ -1 \text{ if input from class B} \end{cases}$$

In these equations η is a positive gain fraction less than 1 and $d(t)$ is the desired correct output for the current input. Note that weights are unchanged if the correct decision is made by the net.

Step 5. Repeat by Going to Step 2